

Rapport de Stratégie – Plateforme MedAssist

Contexte

La plateforme MedAssist gère les dossiers médicaux de **350 cabinets**. Les données sont soumises à la **norme HDS** (Hébergement de Données de Santé) et au **RGPD**. Le SLA contractuel impose une disponibilité de **99,9 %** (≤ 8h44 d'arrêt/an) avec une politique stricte de **zéro perte de données**.

Stack technique : PostgreSQL 16 · Flyway 10 · Docker

1. Analyse des risques par évolution

Évolution	Description	Niveau de risque	Justification
A – Extraction adresses	Création table <code>addresses</code> , migration, suppression colonnes	Moyen	Restructuration d'une table principale mais aucune perte de données possible grâce au pattem dual-write
B – Table doctors	Création table <code>doctors</code> , nettoyage doublons, FK	Moyen	Nettoyage de données texte libre → risque de mauvaise normalisation des noms
C – Extension gender	Migration CHAR(1) → VARCHAR(2), ajout 'NB','U'	Faible	Changement de type simple, valeurs existantes 100 % compatibles
D – Chiffrement SSN	pgcrypto AES-256, suppression clair	Élevé	Opération cryptographique irréversible en phase contract.
E – Partitionnement	Table partitionnée par année RANGE	Élevé	Perte de la clé = perte définitive des SSN Swap de table en production, recréation des FK, risque de perte de données pendant la fenêtre de swap

2. Choix de stratégie : Expand-Contract

Pourquoi Expand-Contract ?

La stratégie **Expand-Contract** (aussi appelée Parallel Change) est la seule qui garantit le **zéro downtime** requis par le SLA 99,9 %. Elle se décompose en 3 phases :

Phase	Action	Downtime	Risque
Expand	Ajout de la nouvelle structure (colonnes, tables) en parallèle de l'existante. Trigger de dual-write pour synchroniser les écritures.	0 sec	Quasi nul – ajout pur
Backfill	Migration des données historiques vers la nouvelle structure. Vérification d'intégrité post-migration.	0 sec	Faible – lecture + écriture sans verrou exclusif
Contract	Suppression de l'ancienne structure après validation complète.	< 5 sec	Moyen – opération irréversible

Alternatives écartées

Stratégie	Raison du rejet
Big Bang (ALTER + UPDATE dans une transaction)	Verrou exclusif sur la table → downtime inacceptable pour 350 cabinets
Blue-Green Database	Complexité excessive, nécessite deux clusters PG synchronisés
Shadow Database	Coût d'infrastructure doublé, latence réseau

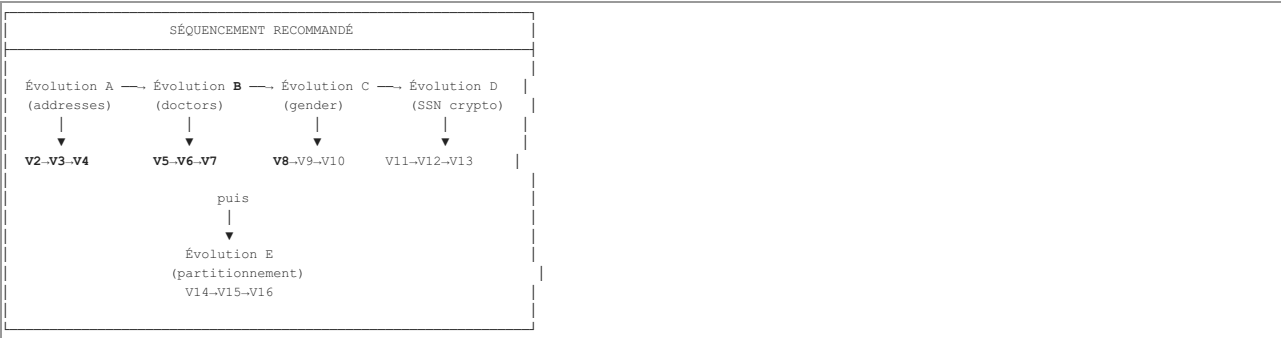
3. Estimation de l'impact (downtime)

Phase	Évolution A	Évolution B	Évolution C	Évolution D	Évolution E
Expand	0 sec	0 sec	0 sec	0 sec	0 sec
Backfill	0 sec	0 sec	0 sec	0 sec	0 sec
Contract	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 5 sec
Total	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 5 sec

Downtime total cumulé estimé : **< 9 secondes** (largement dans le budget SLA de 8h44/an)

4. Séquencement des évolutions

L'ordre d'exécution est critique pour éviter les conflits de schéma :



Justification de l'ordre

- A avant D** : L'évolution A modifie la table `patients`. L'évolution D (chiffrement SSN) modifie aussi `patients`. Les exécuter séquentiellement évite les conflits de verrous et de triggers.
- B avant E** : L'évolution B ajoute `doctor_id` à `consultations`. L'évolution E (partitionnement) doit recréer la structure de `consultations` — elle doit donc inclure `doctor_id`. B doit être entièrement terminée (contract inclus) avant de commencer E.
- C indépendante** : L'évolution C est isolée (modification du type de `gender`). Elle peut s'exécuter à tout moment après A (pour éviter les conflits sur `patients`).
- E en dernier** : Le partitionnement est l'opération la plus risquée (swap de table). Elle doit s'exécuter quand le schéma est stable et finalisé.

5. Détails techniques par évolution

Évolution A – Extraction des adresses

- Expand (V2)** : Création de la table `addresses` avec FK vers `patients`. Trigger AFTER INSERT OR UPDATE sur `patients` pour dual-write vers `addresses`.
- Backfill (V3)** : Migration des données existantes (`address_line1`, `address_line2`, `city`, `postal_code`) vers `addresses`. Vérification par comptage.
- Contract (V4)** : Suppression du trigger, suppression des 4 colonnes d'adresse de `patients`. Création d'une vue `v_patients_with_address` pour la compatibilité applicative.

Évolution B – Table doctors

- Expand (V5)** : Création de `doctors` avec champ `rpps_number` unique. Ajout de `doctor_id` nullable dans `consultations`. Trigger BEFORE INSERT OR UPDATE qui normalise `doctor_name` et

- peuple doctor_id.
- Backfill (V6)** : Extraction des médecins uniques par normalisation (trim, lower, suppression du préfixe "Dr."). Peuplement de doctor_id pour toutes les consultations existantes.
- Contract (V7)** : doctor_id passe en NOT NULL. Suppression de doctor_name. Vue v_consultations_with_doctor pour la compatibilité.

Évolution C – Extension gender

- Expand (V8)** : Ajout de gender_new VARCHAR(2) avec CHECK ('M', 'F', 'NB', 'U'). Trigger de synchronisation gender → gender_new.
- Backfill (V9)** : Copie de gender vers gender_new.
- Contract (V10)** : Suppression de l'ancien gender CHAR(1), renommage gender_new → gender.

Évolution D – Chiffrement SSN

- Expand (V11)** : Activation de pgcrypto. Ajout de ssn_encrypted BYTEA et ssn_hash TEXT (SHA-256 pour l'unicité). Trigger dual-write qui chiffre automatiquement.
- Backfill (V12)** : Chiffrement AES-256 via pgp_sym_encrypt de tous les SSN existants. Vérification par déchiffrement d'un échantillon.
- Contract (V13)** : Suppression de la colonne ssn en clair. NOT NULL sur les colonnes chiffrées. Création d'une fonction sécurisée fn_decrypt_ssn() avec REVOKE PUBLIC.

⚠ **Point critique** : La clé de chiffrement doit être stockée dans un vault sécurisé (HashiCorp Vault, AWS KMS). Sa perte rend les SSN irrécupérables.

Évolution E – Partitionnement

- Expand (V14)** : Création de consultations_partitioned avec PARTITION BY RANGE (consultation_date). 6 partitions (2023-2027 + default). Trigger AFTER INSERT OR UPDATE pour dual-write.
- Backfill (V15)** : Copie des données historiques. Synchronisation de la séquence id. Vérification de comptage.
- Contract (V16)** : Suppression FK de prescriptions. Renommage consultations → consultations_old, consultations_partitioned → consultations. Trigger de validation FK sur prescriptions. Vérification que la table est bien partitionnée via pg_partitioned_table.

⚠ **Note PG16** : Les contraintes UNIQUE sur tables partitionnées doivent inclure la clé de partition. La PK est donc (id, consultation_date). La FK prescriptions → consultations est gérée par trigger au lieu de FK native.

6. Plan de rollback

Chaque version possède un script de rollback dédié (rollback/R_Vxx_*.sql). Le rollback s'exécute en **ordre inverse** :

Rollback V16 → V15 → V14 → V13 → V12 → V11 → V10 → V9 → V8 → V7 → V6 → V5 → V4 → V3 → V2

Les phases **Expand** et **Backfill** sont facilement réversibles (suppression de colonnes/tables ajoutées, vidage de données).

Les phases **Contract** nécessitent une reconstruction depuis les données de la nouvelle structure (ex : déchiffrement du SSN pour restaurer la colonne en clair).

7. Conformité HDS / RGPD

Exigence	Mesure implémentée
Chiffrement au repos (Art. 32 RGPD)	SSN chiffré AES-256 via pgcrypto (Évolution D)
Minimisation des données (Art. 5 RGPD)	Suppression du SSN en clair après chiffrement
Traçabilité	Colonnes created_at / updated_at sur toutes les tables
Droit à l'oubli (Art. 17 RGPD)	ON DELETE CASCADE sur adresses, possibilité de supprimer un patient et toutes ses données
Intégrité	Vérifications post-backfill avec RAISE EXCEPTION en cas d'anomalie
Disponibilité (SLA 99,9%)	Stratégie Expand-Contract → downtime total < 9 secondes

8. Arborescence du projet

```
MedAssist-feras-karim-matheo/
├── docker-compose.yml          # L5 - Stack PG16 + Flyway 10
├── .env                        # Variables d'environnement
├── rapport_strategie.md     # L1 - Ce document
├── flyway/
│   └── sql/
│       ├── V1__initial_schema.sql
│       ├── V2__evolution_A_expand_adresses.sql
│       ├── V3__evolution_A_backfill_adresses.sql
│       ├── V4__evolution_A_contract_adresses.sql
│       ├── V5__evolution_B_expand doctors.sql
│       ├── V6__evolution_B_backfill doctors.sql
│       ├── V7__evolution_B_contract doctors.sql
│       ├── V8__evolution_C_expand_gender.sql
│       ├── V9__evolution_C_backfill_gender.sql
│       ├── V10__evolution_C_contract_gender.sql
│       ├── V11__evolution_D_expand_ssn_encrypt.sql
│       ├── V12__evolution_D_backfill_ssn_encrypt.sql
│       ├── V13__evolution_D_contract_ssn_encrypt.sql
│       ├── V14__evolution_E_expand_partitioning.sql
│       ├── V15__evolution_E_backfill_partitioning.sql
│       └── V16__evolution_E_contract_partitioning.sql
├── rollback/
│   ├── R_V2__rollback_evolution_A_expand.sql
│   ├── R_V3__rollback_evolution_A_backfill.sql
│   ├── R_V4__rollback_evolution_A_contract.sql
│   ├── R_V5__rollback_evolution_B_expand.sql
│   ├── R_V6__rollback_evolution_B_backfill.sql
│   ├── R_V7__rollback_evolution_B_contract.sql
│   ├── R_V8__rollback_evolution_C_expand.sql
│   ├── R_V9__rollback_evolution_C_backfill.sql
│   ├── R_V10__rollback_evolution_C_contract.sql
│   ├── R_V11__rollback_evolution_D_expand.sql
│   ├── R_V12__rollback_evolution_D_backfill.sql
│   ├── R_V13__rollback_evolution_D_contract.sql
│   ├── R_V14__rollback_evolution_E_expand.sql
│   ├── R_V15__rollback_evolution_E_backfill.sql
│   └── R_V16__rollback_evolution_E_contract.sql
└── tests/
    ├── test_evolution_A.sql
    ├── test_evolution_B.sql
    ├── test_evolution_C.sql
    ├── test_evolution_D.sql
    └── test_evolution_E.sql
```